

## DAY 8

**NAME:** Nandhini V

**DATE:** 13-08-2025

### **Task:**

1. Create a dependency injection sample for Painter class
  - 1.1 inject tools (or paintbrush) to painter object via Dependency injection
2. Create at least 10 NUnit tests
  - 2.1 run the tests using both visual studio and command line

### **Code:**

#### **ITool.cs**

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day8Task
{
    public interface ITool
    {
        string Name { get; set; }
        string Use();
    }
}
```

#### **PaintBrush.cs**

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day8Task
{
    public class PaintBrush : ITool
```

```

{
    public string Name { get; set; }

    public PaintBrush(string name)
    {
        Name = name;
    }

    public string Use()
    {
        return $"Painting the wall smoothly with {Name}";
    }
}

```

### **SprayGun.cs**

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day8Task
{
    public class SprayGun : ITool
    {
        public string Name { get; set; }

        public SprayGun(string name)
        {
            Name = name;
        }

        public string Use()
        {
            return $"Spraying the wall evenly with {Name}";
        }
    }
}

```

### **Painter.cs**

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Day8Task
{
    public class Painter
    {
        public string Name { get; set; }
        public ITool Tool { get; set; }

        public Painter(string _name, ITool _tool)
        {
            Name = _name;
            Tool = _tool;
        }

        public string StartPainting()
        {
            return $"{Name} is {Tool.Use()}";
        }

        public void UpgradeTool(bool isAdvanced)
        {
            if (isAdvanced)
            {
                Tool = new SprayGun("Pro Spray");
            }
        }
    }
}

```

## Workshop.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Day8Task
{
    public class Workshop
    {
        public static ITool FetchTool(bool isAdvanced)
        {
            if (isAdvanced)
            {
                return new SprayGun("Advanced Spray Gun");
            }
            else
            {

```

```

        return new PaintBrush("Fine Brush");
    }
}
}

```

## Program.cs

```

namespace Day8Task;

class Program
{
    static void Main(string[] args)
    {
        ITool brush = new PaintBrush("Round Brush");
        ITool spray = new SprayGun("Air Spray");

        Painter painter1 = new Painter("Arun", brush);
        Painter painter2 = new Painter("Riya", spray);

        Console.WriteLine($"{painter1.StartPainting()}");
        Console.WriteLine($"{painter2.StartPainting()}");

        ITool toolFromFactory = Workshop.FetchTool(true);
        Painter painter3 = new Painter("John", toolFromFactory);
        Console.WriteLine($"{painter3.StartPainting()}");
    }
}

```

## TestPaint.cs

```

using Day8Task;
using Moq;

namespace TestProj
{
    public class TestPaint
    {
        [SetUp]
        public void Setup()
        {
        }

        Painter _sut;
        Mock<ITool> mocked_tool;
    }
}

```

```

[SetUp]
public void SetUp()
{
    mocked_tool = new Mock<ITool>();
    mocked_tool.Setup(x => x.Name).Returns("Mock Brush");
    mocked_tool.Setup(x => x.Use()).Returns("Mock painting action");
    _sut = new Painter("Nivin", mocked_tool.Object);
}

[Test]
public void MockedTool_ShouldBeInjected()
{
    Assert.IsNotNull(_sut.Tool, "Painter should have a tool injected via mock");
}

[Test]
public void MockedTool_ShouldReturnCorrectName()
{
    Assert.That(_sut.Tool.Name, Is.EqualTo("Mock Brush"));
}

[Test]
public void MockedTool_ShouldReturnMockedUseAction()
{
    Assert.That(_sut.Tool.Use(), Is.EqualTo("Mock painting action"));
}

[Test]
public void PainterWithMock_ShouldUseInjectedTool()
{
    var result = _sut.StartPainting();
    StringAssert.Contains("Mock painting action", result);
}

[Test]
public void PainterWithMock_ShouldCallUseOnce()
{
    _sut.StartPainting();
    mocked_tool.Verify(x => x.Use(), Times.Once);
}

[Test]
public void NormalPainter_ShouldHavePaintBrush()
{
    var painter = new Painter("Rajesh", new PaintBrush("Fine Brush"));
    Assert.That(painter.Tool.Name, Is.EqualTo("Fine Brush"));
}

[Test]

```

```

public void NormalPainter_ShouldPaintWithCorrectTool()
{
    var painter = new Painter("Kamal", new PaintBrush("Fine Brush"));
    var result = painter.StartPainting();
    StringAssert.Contains("Fine Brush", result);
}

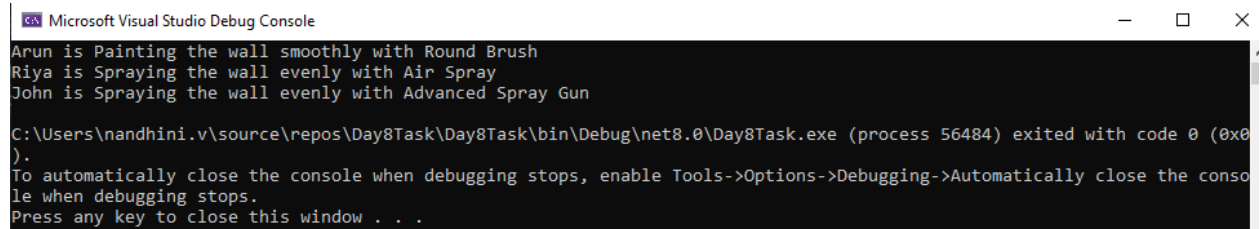
[Test]
public void NormalPainter_ShouldUpgradeToSprayGun()
{
    var painter = new Painter("Rithu", new PaintBrush("Basic Brush"));
    painter.UpgradeTool(true);
    Assert.IsInstanceOf<SprayGun>(painter.Tool);
}

[Test]
public void Workshop_ShouldReturnPaintBrush()
{
    var tool = Workshop.FetchTool(false);
    Assert.IsInstanceOf<PaintBrush>(tool);
}

[Test]
public void Workshop_ShouldReturnSprayGun()
{
    var tool = Workshop.FetchTool(true);
    Assert.IsInstanceOf<SprayGun>(tool);
}
}
}

```

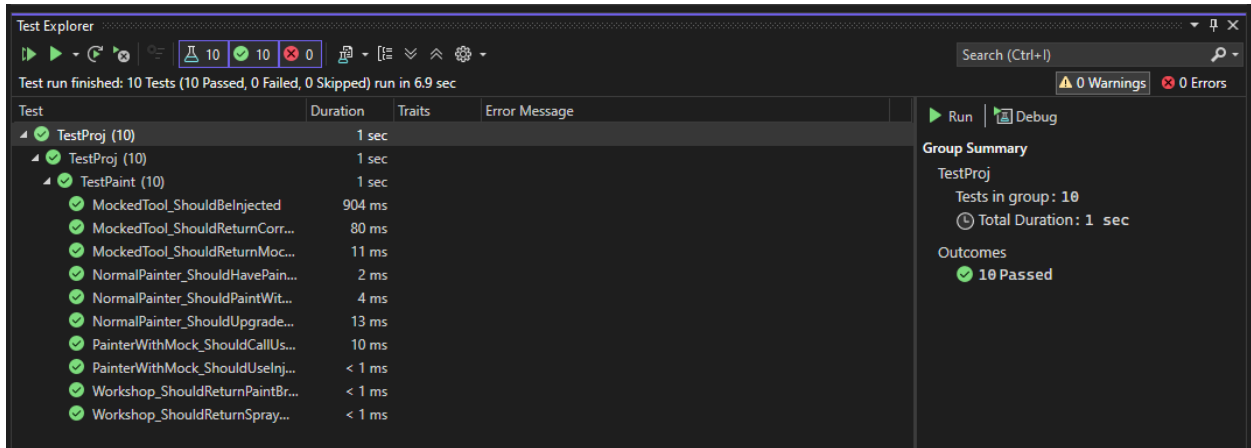
## Output:



```

Microsoft Visual Studio Debug Console
Arun is Painting the wall smoothly with Round Brush
Riya is Spraying the wall evenly with Air Spray
John is Spraying the wall evenly with Advanced Spray Gun
C:\Users\nandhini.v\source\repos\Day8Task\Day8Task\bin\Debug\net8.0\Day8Task.exe (process 56484) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```



## Run via command prompt:

```
Microsoft Windows [Version 10.0.19045.6093]
(c) Microsoft Corporation. All rights reserved.

C:\Users\nandhini.v\source\repos\Day8Task\TestProj>dotnet test TestProj.csproj
Restore complete (1.3s)
  Day8Task succeeded (6.8s) -> C:\Users\nandhini.v\source\repos\Day8Task\Day8Task\bin\Debug\net8.0\Day8Task.dll
  TestProj succeeded (1.6s) -> bin\Debug\net8.0\TestProj.dll
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in C:\Users\nandhini.v\source\repos\Day8Task\TestProj\bin\Debug\net8.0\TestProj.dll
NUnit3TestExecutor discovered 10 of 10 NUnit test cases using Current Discovery mode, Non-Explicit run
NUnit Adapter 4.5.0.0: Test execution complete
  TestProj test succeeded (5.1s)

Test summary: total: 10, failed: 0, succeeded: 10, skipped: 0, duration: 5.1s
Build succeeded in 16.4s

C:\Users\nandhini.v\source\repos\Day8Task\TestProj>
```